



LINGUAGEM DE PROGRAMAÇÃO

AULA 1



Prof. Wellington Rodrigo Monteiro



CONVERSA INICIAL

Olá! Esperamos que esteja tudo bem com você. Uma das frases normalmente comentadas por professores, estudantes e praticantes de TI em qualquer lugar do mundo é a seguinte: “quem domina uma linguagem de programação consegue facilmente dominar outras”. Essa afirmação, naturalmente, não é uma regra: existem linguagens e linguagens de programação. Umas são mais simples, e outras são muito mais complexas. C++ é uma linguagem diferente do Perl, que é diferente do Python, que é diferente do PHP, que é diferente do C#, por exemplo. Por outro lado, poderíamos argumentar que essa frase pode ser compreendida mais ou menos dessa forma: “quando você sabe os conceitos básicos de uma linguagem de programação, como variáveis, estruturas de seleção e/ou repetição, bibliotecas, e sabe ler a documentação, você tem menos dificuldade para entender esses mesmos conceitos em novas linguagens de programação porque agora você já conhece o conceito e já colocou em prática o que aprendeu pelo menos uma vez no passado”.

E é para isso que começamos com Python: é rápido começarmos a criar um algoritmo e, além disso, não há a obrigatoriedade de instalar algo no computador para começar a testar: o Google Colab¹ é uma ótima porta de entrada para isso. Há uma ampla documentação disponível e uma comunidade forte. Além disso, já é utilizada por várias empresas para o desenvolvimento de algoritmos.

Quando aprendemos Python (ou qualquer outra linguagem), geralmente somos apresentados aos conceitos básicos para darmos os primeiros passos. Isso inclui, por exemplo, a entrada e saída de dados (como digitar o seu nome e mostrá-lo logo em seguida com o *input* e *print*), as operações com variáveis (como multiplicar dois valores que um usuário digitou), estruturas de seleção (com o *if*, *elif* e *else*), estruturas de repetição (como o *for*, *while* e *while True*), listas e funções. Mas isso é só o começo: serve para você se **ambientar à base** de uma linguagem de programação em específico.

E é muito provável que alguém (ou você mesmo!) tenha feito a seguinte pergunta: “como consigo fazer um site com isso que aprendi? Como consigo

¹ Disponível em: <<https://colab.research.google.com/>>. Acesso em: 6 dez. 2021.



mostrar um gráfico? Como consigo tirar uma foto com a minha webcam? Como crio um jogo? Como crio um sistema para gerenciar uma loja? Como crio um sistema para armazenar o usuário e senha de um conjunto de usuários em Python?”.

Note que estes exemplos vão além do que vimos até o momento: é necessário, portanto, que saibamos **como colocá-los em prática**. E é por esse motivo que estamos aqui. Vamos lá?

TEMA 1 – EXPANDINDO O NOSSO REPERTÓRIO COM AS BIBLIOTECAS

Vamos sair um pouco do universo de TI e fazermos juntos um pequeno exercício de pensamento: vamos supor que a torneira da sua cozinha começou a vazar água pela parede. E, para ajudar, você nunca trocou uma torneira na vida e não sabe nada de hidráulica. E agora, o que você faz? Temos aqui algumas alternativas:

1. Você compra uma nova torneira e tenta você mesmo fazer a troca, sem pedir ajuda a ninguém. Nisso, descobre que o problema era mais simples do que pensava e acabou gastando dinheiro à toa.
2. Você mesmo resolve tirar a torneira antiga, sem pedir ajuda a ninguém. Com isso, acaba estourando parte da parede e do encanamento, deixando o problema ainda pior. Agora, precisará chamar um profissional e gastar muito mais dinheiro do que imaginava.
3. Você vai ao YouTube e procura por tutoriais de troca de torneiras que foram bem avaliados. Você tenta reproduzir com cuidado o passo a passo e o adapta a suas próprias necessidades. Não fica perfeito no começo, mas você estará mais bem preparado para as próximas vezes.
4. Contrata alguém para fazer o trabalho por você.

1.1 Para que servem as bibliotecas?

E se, ao invés de falarmos de torneiras, imaginássemos que você conseguiu um estágio para uma equipe de tecnologia e lhe deram uma tarefa para criar um gráfico em Python logo de cara? Temos também algumas alternativas:

1. Você tenta criar um código para gerar o gráfico usando somente os *if/elif/else* e *for/while/while True*, listas e funções. Depois de ficar toda a



semana batendo a cabeça, você percebe que ainda não conseguiu criar gráfico algum.

2. Você tenta criar um código para gerar o gráfico usando os *if/elif/else* e *for/while/while True*, listas e funções. Ao pesquisar na internet, você descobre que existem algumas formas para gerar isso. Os exemplos que você pegou funcionam, mas você não consegue integrar ao código que já possui. Logo, você fica sem saber como adaptar o problema.
3. Você pesquisa na internet para ver como é possível fazer isso em Python. Você descobre que existem **várias** formas e com várias combinações e ajustes possíveis.
4. Você diz que é impossível, e é isso aí.

Essa última alternativa, felizmente, não é uma opção: é possível sim! Para responder a isso, vamos pensar em partes. Você já deve ter se deparado no passado com as **funções** em Python. Elas possuem um objetivo muito legal: o reaproveitamento de código. Com as funções, conseguimos colocar dentro de uma caixa fechada uma lógica que precisa ser **reaproveitada**. Observemos o código a seguir, que reajusta o salário de diferentes cargos se eles estiverem acima de um salário-base.

```
salario_base = 3000
salario_analista = 3500
salario_estagiario = 1500
salario_lider = 9000

# reajuste salarial
reajuste = 0.035
if salario_analista > salario_base:
    salario_analista *= reajuste
    print(f'Novo salario: {salario_analista}')

if salario_estagiario > salario_base:
    salario_estagiario *= reajuste
    print(f'Novo salario: {salario_estagiario}')

if salario_lider > salario_base:
    salario_lider *= reajuste
    print(f'Novo salario: {salario_lider}')
```



A lógica utilizada para o reajuste se repete várias vezes, não é mesmo? Logo, podemos **refatorar** o código (ou seja, melhorá-lo sem alterar o resultado esperado) para usar funções (no caso, a função *reajustar_salario*).

```
salario_base = 3000
salario_analista = 3500
salario_estagiario = 1500
salario_lider = 9000

# reajuste salarial
reajuste = 0.035
def reajustar_salario(salario, valor_base, valor_reajuste):
    if salario > valor_base:
        salario *= valor_reajuste
        print(f'Novo salario: {salario}')

    return salario

salario_analista = reajustar_salario(salario_analista, salario_base, reajuste)
salario_estagiario = reajustar_salario(salario_estagiario, salario_base, reajuste)
salario_lider = reajustar_salario(salario_lider, salario_base, reajuste)
```

E se adicionássemos mais 20 cargos? E se precisássemos mudar a lógica para aplicar o reajuste? Com o uso das funções, não é preciso muito esforço: conseguimos mudar a lógica em um lugar só (no caso, dentro da função *reajustar_salario*) e os ajustes funcionarão para todo o código que vier a chamar essa função. Não precisamos também copiar e colar a mesma lógica de **dentro** da função para usarmos em múltiplos lugares: basta chamar a função em si mais uma vez (ou quantas vezes forem necessárias).

No passado, algumas pessoas desenvolveram funções só para converter uma *string* para um valor inteiro (a função *int*²); para converter uma *string* para um valor decimal (a função *float*³); para mostrar algum texto no terminal ou em um notebook de código (a função *print*⁴); para ler algum dado que um usuário venha a digitar (a função *input*⁵), e assim por diante. Note que, no final das

² Disponível em: <<https://docs.python.org/3/library/functions.html#int>>. Acesso em: 6 dez. 2021.

³ Disponível em: <<https://docs.python.org/3/library/functions.html#float>>. Acesso em: 6 dez. 2021.

⁴ Disponível em: <<https://docs.python.org/3/library/functions.html#print>>. Acesso em: 6 dez. 2021.

⁵ Disponível em: <<https://docs.python.org/3/library/functions.html#input>>. Acesso em: 6 dez. 2021.



contas, trabalhamos até o momento com uma **coleção** de funções que várias outras pessoas desenvolveram e que podemos reaproveitar em vários algoritmos. Essas coleções (ou agrupamentos) possuem outro nome: *módulos*, que, por sua vez, módulos são agrupados em **bibliotecas** (ou *libraries*).

1.2 Bibliotecas em Python

Em Python, existem bibliotecas especializadas para inúmeras aplicações: para mostrar gráficos (como é o caso do problema que comentamos anteriormente); para manipular fotos; para manipular vídeos e músicas; para criar jogos; para comunicar com bancos de dados; para criar algoritmos de inteligência artificial, entre outras.

As bibliotecas possuem algumas vantagens, como:

1. Você consegue rapidamente adicionar funcionalidades no seu código que já foram testadas e desenvolvidas por várias outras pessoas.
2. Você consegue testar diferentes implementações que melhor se adaptem às suas necessidades: em Python, por exemplo, existem dezenas de bibliotecas que tratam somente da geração de gráficos. Uma biblioteca que é preferida para outras pessoas pode não necessariamente ser a sua favorita, e vice-versa.
3. Você consegue instalar e desinstalar rapidamente as bibliotecas de acordo com as suas necessidades.

O Python já possui uma biblioteca padrão⁶ e funções como a *input*, *print*, *float* e *int* fazem parte dela. As bibliotecas também podem incluir tipos de dados além das listas, matrizes, *strings*, datas e números. Veremos mais sobre isso no decorrer desta aula.

Saiba mais

Um bom conselho é explorar a documentação das bibliotecas para conhecer suas possibilidades de uso.

Lembra que comentamos que as bibliotecas são divididas em módulos? A biblioteca padrão do Python possui um módulo especializado⁷ somente em

⁶ Disponível em: <<https://docs.python.org/3/library/>>. Acesso em: 6 dez. 2021.

⁷ Disponível em: <<https://docs.python.org/3/library/zipfile.html>>. Acesso em: 6 dez. 2021.



manipulação de arquivos compactados do tipo zip; outro⁸ especializado somente em listar todos os arquivos ou pastas em determinado local; outro⁹ módulo especializado em operações matemáticas; outro¹⁰ módulo especializado em operações estatísticas; e assim por diante. Saber ler essas documentações e encontrar a melhor biblioteca, módulo e função para o que você precisa pode fazer toda a diferença no seu trabalho.

Existem centenas de milhares de bibliotecas disponíveis, que, geralmente são instaladas com o comando *pip*. É possível sabermos quais são as mais utilizadas com base em rankings facilmente encontrados na internet. Uma busca por “pypi download stats” no Google retorna vários desses rankings, por exemplo. Algumas dessas servem para gerar gráficos como o **matplotlib**, **seaborn** e o **plotly**.

TEMA 2 – CRIANDO GRÁFICOS COM PYTHON

Uma das formas de encarar o desenvolvimento de software é pensar que fazemos este trabalho para **resolver problemas**: seja um aplicativo de internet banking, um software que gerencia uma fábrica ou um algoritmo de inteligência artificial que prevê problemas nas vendas. Logo, tão importante quanto desenvolvermos um **código bom é mostrar que estamos resolvendo um problema**.

Naturalmente, essa definição muda muito de caso para caso: um desenvolvedor de back-end ou um administrador de banco de dados raramente vão se deparar com um cenário no qual terão de gerar algum gráfico em Python. Por outro lado, um engenheiro de dados, um cientista de dados ou um analista de dados precisarão fazer isso **muito**. Ou, ainda, pessoas que estejam desenvolvendo pesquisas de natureza quantitativa (o que inclui um mestrado ou doutorado, por exemplo). É claro que existe o Excel (ou qualquer software de planilhas) para criar alguns gráficos rapidamente, mas ele não gera todos os tipos de gráficos de que poderíamos necessitar. Além disso, existem casos em que precisamos nos conectar diretamente a bases de dados ou manipular um grande volume de dados, que o Excel simplesmente não suportaria.

⁸ Disponível em: <<https://docs.python.org/3/library/glob.html>>. Acesso em: 6 dez. 2021.

⁹ Disponível em: <<https://docs.python.org/3/library/math.html>>. Acesso em: 6 dez. 2021.

¹⁰ Disponível em: <<https://docs.python.org/3/library/statistics.html>>. Acesso em: 6 dez. 2021.



2.1 Bibliotecas mais conhecidas

Em Python, três bibliotecas se destacam: o matplotlib, o seaborn e o plotly. Existem várias outras, como o bokeh, yellowbrick e Altair, mas essas três primeiras são as mais conhecidas:

1. O matplotlib¹¹ foi criado em 2003 e se tornou a biblioteca de geração de gráficos mais usada em Python. No entanto, ela é bem complexa: imagine um painel de um grande avião, cheio de botões e configurações. Muitas são as possibilidades, mas também há uma curva de aprendizagem considerável.
2. O seaborn¹² foi criado quase dez anos depois. Ele usa o matplotlib como base, mas simplifica várias das operações. Ele é bem mais simples de usar e pode gerar gráficos mais bonitos.
3. O plotly¹³ (e em específico o *plotly express*) pode ser simples como o seaborn, mas traz uma diferença: a interatividade. Com o plotly é possível criar animações e interações nos gráficos (como passar o mouse e ver mais informações, por exemplo).

2.2 Exemplos

Vamos supor que queiramos criar um gráfico contendo a visualização de diferentes notas e preços de cafés. Temos essas notas e preços armazenados em duas diferentes listas:

```
preco = [5.29, 6.49, 7.49, 7.99, 8.49, 9.99, 11.99]
nota = [5.5, 7.0, 7.5, 8.5, 9.5, 9.0, 9.5]
```

Observe o código utilizado para criar os gráficos utilizando os mesmos dados, mas com bibliotecas diferentes.

¹¹ Disponível em: <<https://matplotlib.org/>>. Acesso em: 6 dez. 2021.

¹² Disponível em: <<https://seaborn.pydata.org/>>. Acesso em: 6 dez. 2021.

¹³ Disponível em: <<https://plotly.com/python/plotly-express/>>. Acesso em: 6 dez. 2021.

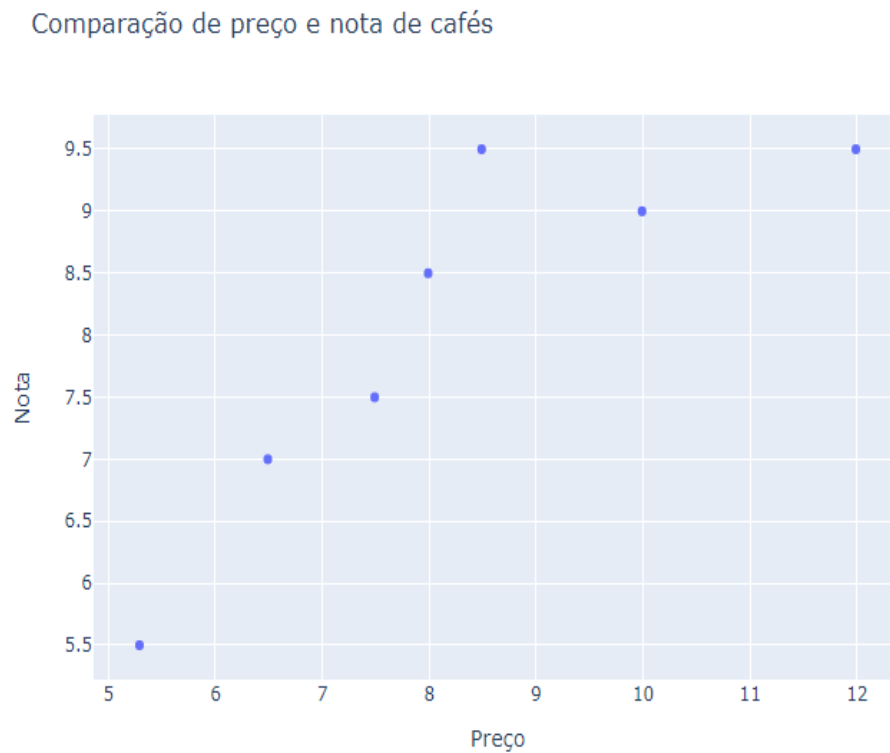


```
import plotly.express as px

fig = px.scatter(x=preco, y=nota,
                 title='Comparação entre preço e nota de cafés',
                 labels={"x": "Preço", "y": "Nota"})

fig.show()
```

Figura 1 – Comparação de preço e nota de cafés (1)



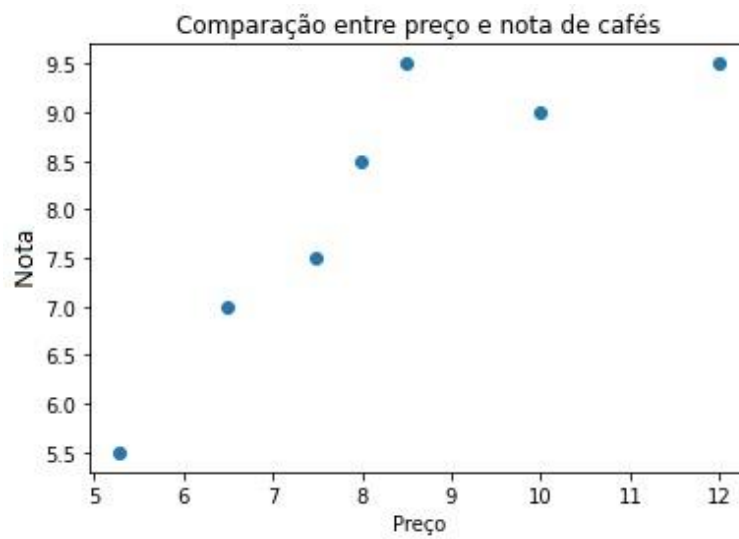
Fonte: Monteiro, 2021.

```
import matplotlib.pyplot as plt

plt.scatter(x=preco, y=nota)
plt.title('Comparação entre preço e nota de cafés')
plt.xlabel('Preço')
plt.ylabel('Nota')
```



Figura 2 – Comparação de preço e nota de cafés (2)



Fonte: Monteiro, 2021.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style('darkgrid')
sns.scatterplot(x=preco, y=nota)
plt.title('Comparação entre preço e nota de cafés')
plt.xlabel('Preço')
plt.ylabel('Nota')
```

Figura 3 – Comparação de preço e nota de cafés (3)

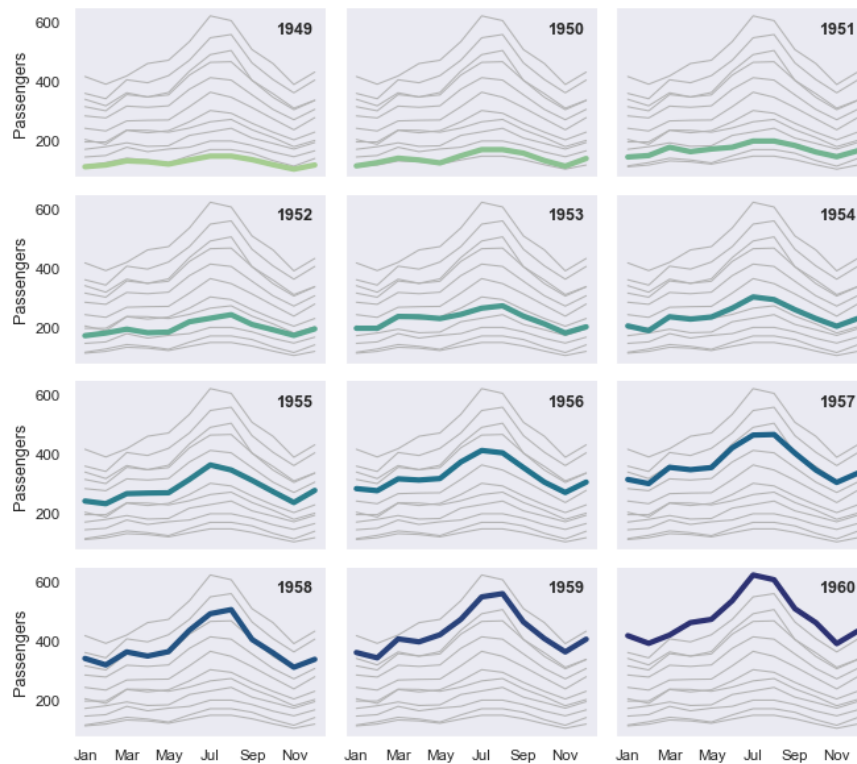


Fonte: Monteiro, 2021.



Esse tipo de gráfico (dispersão, ou *scatter plot*) é um dos mais simples. Por outro lado, existem outras visualizações mais interessantes como esta, do seaborn:

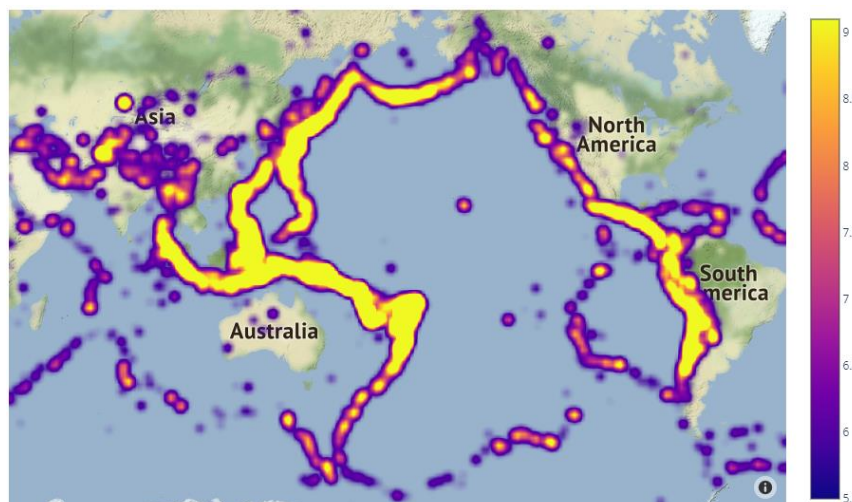
Figura 4 – Seaborn



Fonte: Seaborn, [S.d.].

Ou esta, do Plotly:

Figura 5 – Plotly



Fonte: Plotly Technologies, Inc., 2015.



Naturalmente, para criar gráficos, precisamos de dados. O breve exemplo que criamos anteriormente lida com duas listas: uma chamada *preco* e outra chamada *nota*. Mas será mesmo que o uso de duas listas seria a melhor alternativa? Seria prático permanecermos utilizando listas mesmo se tivéssemos alguns milhares ou milhões de notas? Se você já tentou manipular listas com um tamanho tão alto, deve ter se deparado com problemas de usabilidade e de performance. Felizmente, há uma saída: a biblioteca NumPy.

TEMA 3 – RESOLVENDO O PROBLEMA DAS LISTAS

Várias bibliotecas usadas para inteligência artificial (incluindo redes neurais), visualização, análise de mapas, bioinformática, astronomia e demais áreas utilizam NumPy¹⁴. E isso acontece porque essa biblioteca foi pensada para resolver problemas grandes e complexos com o menor esforço possível por parte do desenvolvedor. Para deixar isso mais claro, sugerimos que voltar ao exemplo do café.

```
preco = [5.29, 6.49, 7.49, 7.99, 8.49, 9.99, 11.99]
nota = [5.5, 7.0, 7.5, 8.5, 9.5, 9.0, 9.5]
```

Cada uma dessas listas possui sete valores e são listas em Python. Agora, imagine fazer as seguintes operações nelas:

1. Multiplicar todos os elementos de uma lista;
2. Encontrar a média e desvio-padrão dos valores de uma lista;
3. Descobrir se algum dos elementos da lista é um número infinito;
4. Obter o valor absoluto de todos os elementos da lista;
5. Obter todos os elementos que sejam somente acima de 9;
6. Remover todos os valores em duplicata;
7. Inverter a ordem de todos os valores.

É bem provável que consigamos fazer rapidamente e com uma só linha de código algumas dessas operações, mas não todas. O NumPy disponibiliza

¹⁴ Disponível em: <<https://numpy.org/>>. Acesso em: 6 dez. 2021.



uma estrutura de dados similar às listas: o *ndarray*¹⁵. O uso do *ndarray* em relação às listas possui algumas vantagens, como:

1. **Funcionalidades:** já existem funções prontas para manipular rapidamente todos os elementos de um *ndarray*. Essas aplicações incluem operações algébricas, estatísticas, manipulações de dados e operações matemáticas. Na prática, essas operações são úteis para pesquisadores a nível de mestrado e doutorado; estudantes em cursos de engenharia e demais cursos aplicados na área de exatas; estatísticos e economistas; desenvolvedores de aplicações back-end que necessitam manipular uma grande massa de dados; cientistas e analistas de dados, entre outros. Logo, o *ndarray* é mais prático do que as listas do Python;
2. **Performance:** quando precisamos manipular uma grande massa de dados (na casa das centenas de milhares de entradas ou mais), o *ndarray* geralmente possui uma implementação mais otimizada. Logo, o *ndarray* tende a ser mais rápido do que as listas do Python;
3. **Consumo de espaço:** além disso, o *ndarray* é também otimizado para grandes conjuntos de dados. Uma prova disso é a de que o uso do NumPy foi essencial para gerar a primeira imagem de um buraco negro conhecida pela humanidade¹⁶. Logo, o *ndarray* consome menos espaço em memória do que as listas do Python;
4. **Reaproveitamento do conhecimento:** toda a base de conhecimento que temos sobre listas é usada para trabalharmos com *ndarray*.

Note como as duas listas anteriores seriam implementadas em *ndarray*:

```
import numpy as np

preco = np.array([5.29, 6.49, 7.49, 7.99, 8.49, 9.99, 11.99])
nota = np.array([5.5, 7.0, 7.5, 8.5, 9.5, 9.0, 9.5])
```

E é isso! Agora, e se precisássemos ler muito mais dados? E se precisássemos ler um arquivo CSV ou uma planilha do Excel? E se quiséssemos

¹⁵ Disponível em: <<https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html>>. Acesso em: 6 dez. 2021.

¹⁶ Disponível em: <<https://numpy.org/case-studies/blackhole-image/>>. Acesso em: 6 dez. 2021.



manipular os dados no Python como se fosse o Excel, o que faríamos? É para isso que serve o **pandas**.

TEMA 4 – TRABALHANDO COM TABELAS EM PYTHON

A manipulação de dados em formato tabular (isto é, em tabelas) é facilitada pela biblioteca pandas. Em conjunto com ela, outras bibliotecas de cunho estatístico e gráfico podem ser combinadas para analisar os dados, manipular os dados, gerar correlações e relatórios de estatística descritiva.

4.1 Pandas

O pandas¹⁷ é uma biblioteca que foi criada utilizando o NumPy como base. Da mesma forma que uma faca é uma ferramenta que todo cozinheiro possui e um alicate é uma ferramenta que todo eletricitista possui, o pandas é uma biblioteca¹⁸ que todo desenvolvedor que atua com dados em Python conhece e trabalha.

O pandas introduz um tipo de dados muito poderoso chamado *DataFrame*¹⁹. Para falar do *DataFrame*, vamos voltar ao exemplo do café. Lembre-se que tínhamos duas listas, *preco* e *nota*:

```
preco = [5.29, 6.49, 7.49, 7.99, 8.49, 9.99, 11.99]
nota = [5.5, 7.0, 7.5, 8.5, 9.5, 9.0, 9.5]
```

Nunca deixamos isso explícito, mas **é possível entender** que o primeiro valor do preço corresponde ao primeiro valor da nota; o segundo valor do preço corresponde ao segundo valor da nota; e assim sucessivamente. Agora, e se eu quisesse ordenar de forma decrescente o preço, alterando também a ordem da nota? Da forma que está, isso não seria possível. E se quisesse descobrir rapidamente o preço correspondente à mediana das notas? Também não seria tão simples. Por outro lado, em Python também podemos ter matrizes (isto é, listas de listas).

¹⁷ Disponível em: <<https://pandas.pydata.org/>>. Acesso em: 6 dez. 2021.

¹⁸ Disponível em: <<https://pandas.pydata.org/docs/index.html>>. Acesso em: 6 dez. 2021.

¹⁹ Disponível em: <<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>>. Acesso em: 6 dez. 2021.



Então, poderíamos organizar da seguinte forma:

```
precos_notas = [[5.29, 6.49, 7.49, 7.99, 8.49, 9.99, 11.99],  
                [5.5, 7.0, 7.5, 8.5, 9.5, 9.0, 9.5]]
```

Agora temos somente uma variável, chamada *precos_notas*. Mas e se quiséssemos incluir a data da coleta das notas? Ou, ainda, se quiséssemos aplicar uma operação para *todas* as notas (um pequeno reajuste de 0.1 pontos, talvez)? Ou, quem sabe, se quiséssemos **dar um nome** para esses dados, como seria?

É para responder a essas questões que existem os DataFrames. Com uma linha (na maioria das vezes), conseguimos ler planilhas e conjuntos de dados inteiros para uma variável, e vice-versa. Quer ler uma tabela em SQL e converter para um DataFrame? É possível. Quer ler uma planilha do Excel? Dá sim! Um JSON, quem sabe? É claro! As possibilidades são múltiplas. Vamos ver como ficariam os mesmos dados dos preços e das notas, mas em DataFrame:

```
import pandas as pd  
  
precos_notas = pd.DataFrame([[5.29, 5.5], [6.49, 7.0], [7.49, 7.5], [7.99, 8.5],  
                             [8.49, 9.5], [9.99, 9.0], [11.99, 9.5]],  
                             columns=['Preço', 'Nota'])
```

Tabela 1 – Preço e nota

	Preço	Nota
0	5.29	5.5
1	6.49	7.0
2	7.49	7.5
3	7.99	8.5
4	8.49	9.5
5	9.99	9.0
6	11.99	9.5



E se quiséssemos criar uma data fictícia para a coleta dessas notas?

```
precos_notas['DataColeta'] = pd.date_range(start='2022-01-01', freq='M',  
                                           periods=len(precos_notas))
```

A função *date_range*²⁰ cria uma sequência de datas. Podemos informar uma data de início e de fim ou, ainda, uma frequência entre as datas. No caso, preferimos escolher somente uma data de início (2022-01-01) e uma frequência mensal, pegando sempre o último dia de cada mês (com o parâmetro 'M'). A quantidade de datas foi definida pela quantidade de linhas disponíveis no dataframe (pelo *len(precos_notas)*).

Tabela 2 – Preço, nota e data

	Preço	Nota	DataColeta
0	5.29	5.5	2022-01-31
1	6.49	7.0	2022-02-28
2	7.49	7.5	2022-03-31
3	7.99	8.5	2022-04-30
4	8.49	9.5	2022-05-31
5	9.99	9.0	2022-06-30
6	11.99	9.5	2022-07-31

E se quiséssemos criar colunas para extrair somente o ano, ou somente o dia, ou somente o dia da semana da data de coleta? Ou, ainda, a proporção da nota em relação ao preço? Vamos lá:

```
precos_notas['Ano'] = precos_notas['DataColeta'].dt.year  
precos_notas['Dia'] = precos_notas['DataColeta'].dt.day  
precos_notas['DiaSemana'] = precos_notas['DataColeta'].dt.weekday  
precos_notas['Proporcao'] = precos_notas['Preço']/precos_notas['Nota']
```

²⁰ Disponível em: <https://pandas.pydata.org/docs/reference/api/pandas.date_range.html>. Acesso em: 6 dez. 2021.



Agora, vamos pensar um pouco: já imaginou como seria fazer tudo isso sem o pandas, somente manipulando as listas em Python? Há ainda uma função no pandas para nos fornecer dados estatísticos sobre esse DataFrame: o *describe*²¹. Observe:

```
precos_notas.describe()
```

Tabela 3 – Preço, nota, data e proporção

	Preço	Nota	DataColeta	Ano	Dia	DiaSemana	Proporcao
0	5.29	5.5	2022-01-31	2022	31	0	0.961818
1	6.49	7.0	2022-02-28	2022	28	0	0.927143
2	7.49	7.5	2022-03-31	2022	31	3	0.998667
3	7.99	8.5	2022-04-30	2022	30	5	0.940000
4	8.49	9.5	2022-05-31	2022	31	1	0.893684
5	9.99	9.0	2022-06-30	2022	30	3	1.110000
6	11.99	9.5	2022-07-31	2022	31	6	1.262105

Aqui, temos dados como a quantidade de valores para cada coluna (*count*), a média (*mean*), o desvio-padrão (*std*), os valores mínimo (*min*) e máximo (*max*), a mediana (50%) e os quartis inferior (25%) e superior (50%). Além disso, veja que na própria documentação²² existem ainda múltiplos exemplos para operações, como agrupamento, deslocamento de colunas, ordenação, preenchimento de valores nulos, remoção de linhas repetidas, arredondamento, correlação, entre outros. Falando em correlação, vamos testar uma coisa.

4.2 Estudando correlação com diferentes bibliotecas

Quando falamos em **correlação**, queremos, na prática, entender o quanto uma variável afeta outra. Por exemplo, será que, toda vez que o preço aumenta

²¹ Disponível em: <<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.describe.html>>. Acesso em: 6 dez. 2021.

²² Disponível em: <<https://pandas.pydata.org/pandas-docs/stable/reference/frame.html>>. Acesso em: 6 dez. 2021.



em 10%, a nota também aumenta (ou diminui) na mesma proporção? Com uma única linha, o pandas consegue calcular isso²³. Veja só:

```
precos_notas.corr(method='spearman')
```

Tabela 4 – Correlação

	Preço	Nota	Ano	Dia	DiaSemana	Proporcao
Preço	1.000000	0.936975	NaN	0.119523	0.763763	0.464286
Nota	0.936975	1.000000	NaN	0.241209	0.642229	0.180187
Ano	NaN	NaN	NaN	NaN	NaN	NaN
Dia	0.119523	0.241209	NaN	1.000000	0.152145	0.239046
DiaSemana	0.763763	0.642229	NaN	0.152145	1.000000	0.600099
Proporcao	0.464286	0.180187	NaN	0.239046	0.600099	1.000000

Quanto mais próximo de 1, maior a correlação diretamente proporcional (ou seja, quando um aumenta, o outro aumenta também). Quanto mais próximo de -1, maior a correlação inversamente proporcional (ou seja, quando um aumenta, o outro diminui). Aparentemente, o preço tem uma correlação muito alta com a nota, mas parece que o dia da semana também teria uma correlação interessante. Ficou difícil de enxergar vários números? Vamos ao seaborn²⁴:

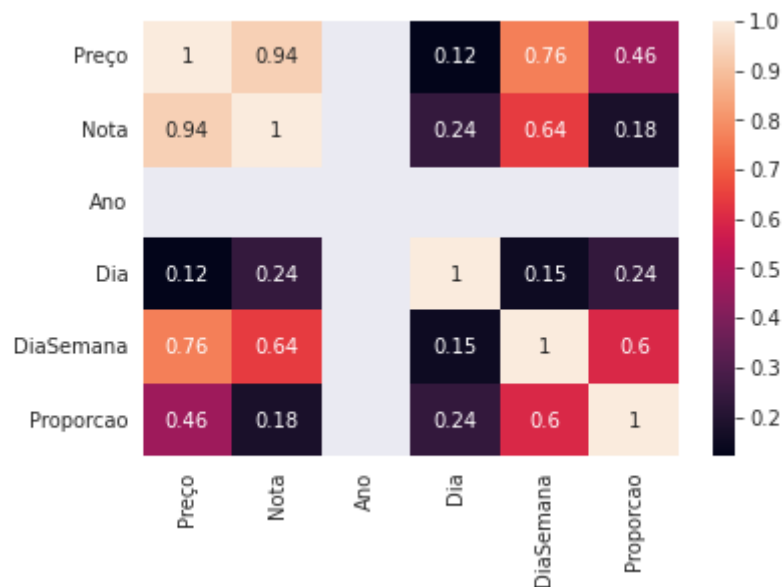
```
sns.heatmap(precos_notas.corr(method='spearman'), annot=True)
```

²³ Disponível em: <<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.corr.html#pandas.DataFrame.corr>>. Acesso em: 6 dez. 2021.

²⁴ Disponível em: <<https://seaborn.pydata.org/generated/seaborn.heatmap.html>>. Acesso em: 6 dez. 2021.



Figura 6 – Seaborn



Perceba que, aos poucos, vamos também **misturando** bibliotecas. A ideia aqui é conhecermos essas diferentes bibliotecas para saber que possibilidades elas nos trazem. E, então, saberemos como aproveitar o melhor de cada uma para atender às nossas necessidades. É como se fôssemos um grande chef de cozinha que mistura diferentes técnicas que aprendeu ao longo da vida para construir um belo prato. Ele não ficará restrito aos instrumentos e temperos que estão à sua frente, mas pensará além, para integrar diferentes técnicas, utensílios e temperos para alcançar algo novo, inovador e que atenda às suas necessidades. Aqui, a lógica é a mesma (ainda que não seja tão saborosa). É por isso que apresentamos diferentes bibliotecas e sua documentação; é importante que você conheça o que existe disponível para o seu uso, que consiga ter a autonomia de encontrar a informação de que precisa e o desejo de ir além, de descobrir coisas novas e experimentar diferentes funções e bibliotecas.

TEMA 5 – RESOLVENDO PROBLEMAS COM UMA LINHA: USO DE EXPRESSÕES REGULARES EM PYTHON

Agora que conversamos sobre **outras** bibliotecas, vamos retornar à biblioteca-padrão do Python. E, para isso, vamos conversar sobre uma ferramenta poderosa: as **expressões regulares**.



Para falarmos de expressões regulares (*regular expressions*, ou *regex*), vamos, primeiramente, pensar no seguinte desafio: vamos supor que a empresa na qual você trabalha precisa processar um arquivo contendo o equivalente a 500 páginas de texto, e lhe deram a tarefa de selecionar **somente** as placas de veículos.

Agora, o que seria uma placa válida? Até alguns anos atrás, uma placa válida possuía três letras e quatro dígitos. Logo, as seguintes placas seriam combinações válidas:

1. ABC1234.
2. ABC-1234.

Note que o segundo exemplo possui um hífen: existem pessoas que se referem às placas com o hífen e outras, sem. Agora, as seguintes combinações não são placas válidas:

1. 12345678.
2. ABC12.
3. 1234ABC.
4. A-B-C-1234.
5. ABCDE1234.
6. A1234BC.

Note que essas combinações não respeitam o **padrão** das placas que comentamos anteriormente: três letras seguidas de quatro dígitos. Mas, recentemente, essa regra mudou: também passaram a ser válidas placas cujo quinto caractere é uma letra (conhecido como *padrão Mercosul*), como:

1. ABC1A34.
2. ABC-1A34.

Logo, note que temos quatro **padrões** a que precisaríamos atender, pelo menos:

1. Três primeiros caracteres sendo letras e os quatro caracteres seguintes sendo dígitos (ABC1234).
2. Três primeiros caracteres sendo letras, seguidos de um hífen e, em seguida, os próximos quatro caracteres sendo dígitos (ABC-1234).



3. Três primeiros caracteres sendo letras; o quarto caractere sendo um dígito; o quinto sendo uma letra; e o sexto e sétimo sendo dígitos (ABC1A34);
4. Três primeiros caracteres sendo letras, seguidas de um hífen; o quarto caractere sendo um dígito; o quinto sendo uma letra; e o sexto e sétimo sendo dígitos (ABC-1A34).

Observe que realçamos a palavra **padrão** anteriormente: não queremos somente detectar a placa “ABC1234”, mas qualquer outra placa que siga esse exemplo (ZZZ9999, UNI0000, AAA0800 etc.). Isso dá mais de 170 milhões de combinações. Se considerarmos as placas do padrão Mercosul, temos quase 500 milhões de combinações a mais para encontrar.

E para colocar tudo isso em um algoritmo? Já imaginou ter que digitar todas as combinações possíveis? Ainda que tentássemos usar um loop (como o *for*) ou algo do gênero, teríamos que percorrer todas as alternativas possíveis, o que demandaria um considerável esforço nosso e do computador, para dar conta de todo o esforço.

É para isso que serve o regex: criamos uma string contendo **somente** o padrão que desejamos detectar (e, às vezes, o que **não queremos** detectar). Existem listas bem extensas sobre as possibilidades do regex, da Mozilla²⁵, da Microsoft²⁶ ou, ainda, do próprio Python²⁷. Observe alguns exemplos:

Quadro 1 – Regex

Regex	O que faz?	Exemplo	O que seleciona?
*	Seleciona o elemento anterior nenhuma ou mais vezes	Caf*	“Ca” em “Caça” “Caf” em “Café” “Caffff” em “Caffffé”
+	Seleciona o elemento anterior uma ou mais vezes	Caf+	Nada em “Caça” “Caf” em “Café” “Caffff” em “Caffffé”

²⁵ Disponível em: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions/Cheatsheet>. Acesso em: 6 dez. 2021.

²⁶ Disponível em: <<https://docs.microsoft.com/en-us/dotnet/standard/base-types/regular-expression-language-quick-reference>>. Acesso em: 6 dez. 2021.

²⁷ Disponível em: <<https://docs.python.org/3/howto/regex.html>>. Acesso em: 6 dez. 2021.



?	Seleciona o elemento anterior nenhuma ou uma vez	Caf?	"Ca" em "Caça" "Caf" em "Café" "Caf" em "Caffffé"
{n}	Seleciona o elemento anterior n vezes	Caf{2}	Nada em "Caça" "Caf" em "Café" "Caff" em "Caffffé"
{n,m}	Seleciona o elemento anterior de n até m vezes	Caf{1,2}	Nada em "Caça" "Caf" em "Café" "Cafff" em "Caffffé"
[qualquer_caractere]	Seleciona qualquer caractere que esteja em parênteses. "A-Z" selecionará todos os caracteres de A (maiúsculo) até Z (maiúsculo), por exemplo.	[a-z]+	As duas letras "a" em "Caça" "af" em "Café" "affff" em "Caffffé"

Imaginemos a frase *"The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually."*. Veja, no quadro a seguir, quais padrões cada regex detecta na frase. Cada mudança de cor representa uma sequência que foi encontrada.

Quadro 2 – Utilização de regex

Regex	O que seleciona?
[0-9]	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[0-9]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[\d]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[a-z]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.



[A-Z]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[A-Z0-9]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[A-Za-z]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[\w]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[A-Za-z0-9]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[A-Za-z0-9_]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[\w\d]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
\s	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[\w\d\s]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
[^\.]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.
\w[^\.]+	The quick brown Fox jumps over the lazy dog. 1 or 1000 foxes, actually.

E aquele caso das placas de automóveis? Observemos a frase “3 placas devem ser selecionadas: UNI0100, NTE-0A00 e RRR9876. A JJ2070 é inválida.”.



Quadro 3 – Placas de automóveis

Regex	O que seleciona?
<code>[A-Za-z0-9]{7}</code>	3 placas devem ser selecionadas: UNI0100 , NTE-0a00 e RRR9876. A JJ2070 é inválida.
<code>[A-Za-z]{3}[0-9]+</code>	3 placas devem ser selecionadas: UNI0100 , NTE-0a00 e RRR9876. A JJ2070 é inválida.
<code>[A-Za-z]{3}[0-9]+</code>	3 placas devem ser selecionadas: UNI0100 , NTE-0a00 e RRR9876. A JJ2070 é inválida.
<code>[A-Za-z]{3}\d{4}</code>	3 placas devem ser selecionadas: UNI0100 , NTE-0a00 e RRR9876. A JJ2070 é inválida.
<code>[A-Za-z]{3}\-?\d[\dA-Za-z]\d{2}</code>	3 placas devem ser selecionadas: UNI0100 , NTE-0a00 e RRR9876. A JJ2070 é inválida.
<code>[A-Za-z]{3}\-?\d[A-Ja-j\d]\d{2}</code>	3 placas devem ser selecionadas: UNI0100 , NTE-0a00 e RRR9876. A JJ2070 é inválida.

Logo, o uso de regex é bem interessante para encontrar rapidamente múltiplos resultados dentro de um conjunto grande de textos. Ele possui algumas aplicações úteis, como:

1. Busca em textos por palavras-chave ou padrões.
2. Validação de dados (o CEP que o usuário digitou é realmente um CEP válido? O e-mail digitado realmente é um email?).
3. Web scraping (como a extração de somente os títulos das notícias em um portal de notícias).
4. Filtros em integrações (como a obtenção de somente os dados que precisamos a partir de uma integração construída com outro servidor ou com um banco de dados).



Em Python, o módulo a cargo do regex é o módulo `re`²⁸. O código a seguir é um exemplo de implementação do regex de busca de placas que vimos anteriormente.

```
import re

# texto que usaremos na busca
texto_busca = '3 placas devem ser selecionadas: UNI0100, NTE-0a00 e RRR9876. A JJ2070 é inválida.'

# regex a ser usado
texto_regex = '[A-Za-z]{3}\-?\d[A-Ja-j\d]\d{2}'

# procurando
resultados = re.findall(texto_regex, texto_busca)
resultados
```

Caso tenha interesse, sugerimos que dê uma olhada nos links do módulo `re` do Python, bem como nos links sobre regex sobre os quais já comentamos. Saber usar regex pode fazer toda a diferença para resolver rapidamente problemas que seriam complexos no seu dia a dia. Caso também queira testar diferentes tipos de regex, existem sites que podem ajudar você a ver como funciona o regex de forma visual. Alguns exemplos incluem o `regex101`²⁹, o `RegExr`³⁰ e o `Debuggex`³¹.

Saiba mais

Geralmente testamos um regex em algum desses sites para ver se a lógica funciona. Somente após garantirmos que o regex está selecionando o que gostaríamos (e não selecionando o que não gostaríamos) é que colocamos a lógica no código.

²⁸ Disponível em: <<https://docs.python.org/3/library/re.html>>. Acesso em: 6 dez. 2021.

²⁹ Disponível em: <<https://regex101.com/>>. Acesso em: 6 dez. 2021.

³⁰ Disponível em: <<https://regexr.com/>>. Acesso em: 6 dez. 2021.

³¹ Disponível em: <<https://www.debuggex.com/>>. Acesso em: 6 dez. 2021.



FINALIZANDO

Observe que a intenção desta aula foi mostrar a você a relevância do uso de outras bibliotecas em Python: na realidade, este é um dos motivos pelos quais o Python acaba tendo uma preferência por várias empresas, profissionais e estudantes. Isso naturalmente não significa que o Python é a **melhor** linguagem do mundo, cada linguagem de programação possui os seus prós e os seus contras. Em nosso contexto, a oportunidade de conhecermos diferentes bibliotecas é muito útil para sabermos como podemos **rapidamente** aplicar a linguagem de programação em novos desafios, algoritmos e soluções de mercado.

Nesse sentido, observe que comentamos sobre o uso de **expressões regulares**, **gráficos** e dados tabulares com o **NumPy** e **pandas**. Também comentamos sobre outras bibliotecas que podem igualmente ser úteis para o seu conhecimento. Independentemente da biblioteca, observe que também disponibilizamos links para você ter acesso à documentação em sua versão mais atualizada. Saber ler e compreender a documentação é certamente algo que lhe dará vantagem competitiva no mercado de trabalho. Logo, entenda as discussões que tivemos até o momento como direcionadores. Apontamos os diferentes pontos de partida que são do seu interesse, e não um passo a passo – até porque esse passo a passo muda a cada desafio no trabalho. Assim, é importante que você saiba como raciocinar para executar, e não como executar um passo a passo. Novamente: leia, explore e teste as documentações.



REFERÊNCIAS

AGUILAR, L. J. **Fundamentos de programação**: algoritmos, estruturas de dados e objetos. 3. ed. Porto Alegre: AMGH, 2011.

GRUS, J. **Data Science do Zero**. Rio de Janeiro: Editora Alta Books, 2021.

PERKOVIC, L. **Introdução à Computação Usando Python**: um Foco no Desenvolvimento de Aplicações. Rio de Janeiro: LTC, 2016.

PLOTLY TECHNOLOGIES, INC. **Collaborative data science**. Montréal, 2015.

SEABORN. **Small multiple time series**. [S.d.]. Disponível em: <https://seaborn.pydata.org/examples/timeseries_facets.html>. Acesso em: 6 dez. 2021.

WASKOM, M. L. Seaborn: statistical data visualization. **Journal of Open Source Software**, v. 6, n. 60, p. 3021, 2021.